

15 mai 2026

Table des matières

Préface

La programmation est devenue un outil indispensable pour tout scientifique. Que vous étudiez les mathématiques, la physique, la chimie ou la biologie, savoir écrire un programme vous permet d'automatiser des calculs, de visualiser des données, de simuler des phénomènes et de tester des hypothèses.

Ce cours est conçu pour des étudiants de Licence **sans aucune expérience préalable en programmation**. Nous partons de zéro et progressons pas à pas, en illustrant chaque concept par des exemples issus des sciences. L'objectif est de vous rendre autonome pour résoudre des problèmes scientifiques avec Python (et une introduction à R).

Prérequis. Aucun en programmation. Des notions de mathématiques de lycée (fonctions, équations) sont utiles mais pas indispensables.

Philosophie. Chaque chapitre suit le même schéma :

1. Explication du concept avec une analogie scientifique.
2. Code commenté avec sa sortie.
3. Erreurs courantes (« Ce qui peut mal tourner »).
4. Mini-projet scientifique.
5. Exercices gradués.

Références.

- DOWNEY — *Think Python*, O'Reilly (disponible gratuitement en ligne).
- LANGTANGEN — *A Primer on Scientific Programming with Python*, Springer.
- VANDERPLAS — *Python Data Science Handbook*, O'Reilly (gratuit en ligne).

Chapitre 0 — Installation et Configuration

Avant de commencer à programmer, il faut préparer votre environnement de travail. Ce chapitre vous guide pas à pas.

Installer Python

Intuition

Python est un **langage de programmation** gratuit, utilisé par des millions de scientifiques dans le monde. C'est comme une calculatrice surpuissante qui comprend vos instructions écrites en anglais.

Méthode recommandée : Anaconda

Anaconda est une distribution Python qui inclut toutes les bibliothèques scientifiques dont nous aurons besoin.

1. Rendez-vous sur <https://www.anaconda.com/download>
2. Téléchargez la version pour votre système (Windows, macOS ou Linux).
3. Lancez l'installateur et suivez les instructions par défaut.
4. Vérifiez l'installation en ouvrant un terminal et en tapant :

Terminal

```
<MINTED>
```

Sortie

```
Python 3.12.4
```

Alternative légère : Python seul + pip

Si vous préférez une installation minimale :

1. Téléchargez Python depuis <https://www.python.org/downloads/>
2. Cochez « Add Python to PATH » lors de l'installation.
3. Installez les bibliothèques nécessaires :

Terminal

<MINTED>

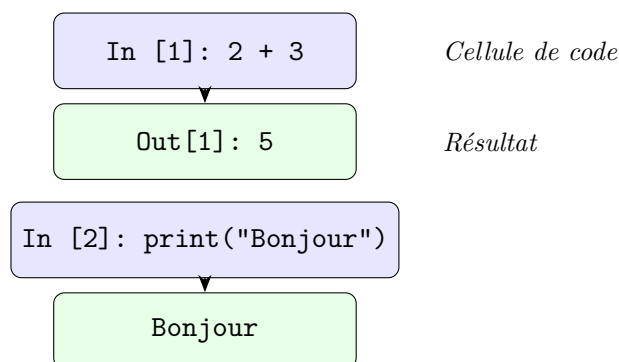
Jupyter Notebook

Jupyter Notebook est un environnement interactif où vous écrivez du code, voyez les résultats et ajoutez des notes — comme un cahier de laboratoire numérique.

Terminal — Lancer Jupyter

<MINTED>

Votre navigateur s'ouvre automatiquement. Cliquez sur **New** → **Python 3** pour créer un nouveau notebook.



Bonne pratique

Utilisez **Shift + Entrée** pour exécuter une cellule et passer à la suivante. Utilisez **Ctrl + Entrée** pour exécuter sans changer de cellule.

Éditeurs de texte

Pour écrire des scripts Python (fichiers `.py`), vous pouvez utiliser :

- **VS Code** (recommandé) : gratuit, avec extension Python.
- **Spyder** : inclus dans Anaconda, conçu pour les scientifiques.
- **PyCharm** : version Community gratuite.

Vérification de l'installation

Python — Premier test

<MINTED>

Sortie

Installation réussie !
NumPy version : 1.26.4
Graphique sauvegardé !

Si tout fonctionne, vous êtes prêt(e) à commencer le chapitre 1 !

Installer R (pour le chapitre 9)

1. Téléchargez R depuis <https://cran.r-project.org/>
2. Installez **RStudio Desktop** depuis <https://posit.co/download/rstudio-desktop/>
3. Vérifiez en ouvrant RStudio et en tapant :

Console R

<MINTED>

Sortie

```
[1] "R version 4.4.1 (2024-06-14)"
```

Ce qui peut mal tourner

- « **Python n'est pas reconnu** » : vous avez oublié d'ajouter Python au PATH. Réinstallez en cochant l'option.
- « **ModuleNotFoundError** » : la bibliothèque n'est pas installée. Tapez `pip install nom_du_module`.
- **Permission refusée** : sur macOS/Linux, essayez `pip install --user nom_du_module`.

Chapitre 1

Introduction à la Programmation

1.1 Pourquoi programmer ?

Intuition

Un programme informatique est une **recette de cuisine** pour l'ordinateur. Tout comme une recette décrit étape par étape comment préparer un plat, un programme décrit étape par étape comment résoudre un problème.

En sciences, programmer permet de :

- **Automatiser** des calculs répétitifs (calculer 10 000 intégrales).
- **Visualiser** des données (tracer l'évolution d'une population).
- **Simuler** des phénomènes (mouvement d'un pendule, diffusion de chaleur).
- **Analyser** de grandes quantités de données (génomique, astronomie).

1.2 Pourquoi Python ?

Lisible
Syntaxe proche
de l'anglais

Gratuit
Open source
Multiplateforme

Puissant
NumPy, SciPy
Machine Learning

Populaire
NASA, CERN
Google, Netflix

1.3 Votre premier programme

Python

<MINTED>

Sortie

```
Bonjour, futur(e) scientifique !
```

Félicitations! Vous venez d'écrire votre premier programme. La fonction `print` affiche du texte à l'écran.

1.4 Python comme calculatrice

Python peut effectuer toutes les opérations mathématiques de base :

Python — Calculs de base

```
<MINTED>
```

Sortie

```
8
6
42
3.75
3
3
1024
```

Opérateurs arithmétiques

Opérateur	Signification	Exemple
<code>+</code>	Addition	<code>8 + 0</code> = 8
<code>-</code>	Soustraction	<code>6 - 0</code> = 6
<code>*</code>	Multiplication	<code>6 * 7</code> = 42
<code>/</code>	Division réelle	<code>14 / 4</code> = 3.75
<code>//</code>	Division entière	<code>12 // 4</code> = 3
<code>%</code>	Modulo (reste)	<code>13 % 4</code> = 1
<code>**</code>	Puissance	<code>2 ** 10</code> = 1024

1.5 Commentaires

Les **commentaires** sont des notes pour les humains. Python les ignore complètement.

Python

```
<MINTED>
```

Sortie

```
5
Vitesse de la lumière : 299792458 m/s
```

Bonne pratique

Commentez votre code pour expliquer **pourquoi** vous faites quelque chose, pas **ce que** vous faites. Un bon commentaire dit « Vitesse de la lumière en m/s », pas « On affecte 299792458 à c ».

1.6 Exemple scientifique : Énergie cinétique

Calculons l'énergie cinétique $E_c = \frac{1}{2}mv^2$ d'un objet :

Python — Énergie cinétique

<MINTED>

Sortie

Masse : 75.0 kg
Vitesse : 10.0 m/s
Énergie : 3750.0 J

1.7 Les erreurs : ne pas avoir peur !

Ce qui peut mal tourner

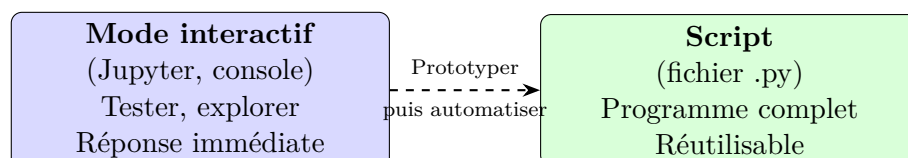
Les erreurs sont **normales** et font partie de l'apprentissage. Python vous aide en indiquant le type d'erreur et la ligne concernée.

Python — Erreurs courantes

<MINTED>

Règle d'or : lisez toujours le message d'erreur **de bas en haut**. La dernière ligne vous dit ce qui ne va pas.

1.8 Le mode interactif vs les scripts



1.9 Mini-projet : Conversions d'unités

Mini-projet scientifique

Écrivez un programme qui convertit des températures entre Celsius et Fahrenheit.

Formules : $F = \frac{9}{5}C + 32$ et $C = \frac{5}{9}(F - 32)$

Python — Convertisseur

<MINTED>

Sortie

100°C = 212.0°F

72°F = 22.2°C

--- Table de conversion ---

°C	°F
0	32.0
10	50.0
20	68.0
30	86.0
40	104.0
50	122.0
60	140.0
70	158.0
80	176.0
90	194.0
100	212.0

1.10 Exercices

Exercice 1.1 (★ — Guidé). Calculez l'aire d'un cercle de rayon $r = 5$ en utilisant la formule $A = \pi r^2$. Utilisez <MINTED> pour π .

Exercice 1.2 (★ — Guidé). Calculez la distance entre deux points $(x_1, y_1) = (1, 2)$ et $(x_2, y_2) = (4, 6)$ avec la formule $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Indice : $\sqrt{x} = x^{0.5}$.

Exercice 1.3 (★★ — Indépendant). La loi de la gravitation universelle donne la force entre deux masses : $F = G \frac{m_1 m_2}{r^2}$, avec $G = 6.674 \times 10^{-11} \text{ N}\cdot\text{m}^2/\text{kg}^2$. Calculez la force entre la Terre ($m_1 = 5.972 \times 10^{24} \text{ kg}$) et la Lune ($m_2 = 7.342 \times 10^{22} \text{ kg}$), distantes de $r = 3.844 \times 10^8 \text{ m}$.

Exercice 1.4 (★★★ — Défi scientifique). Calculez la longueur d'onde de De Broglie $\lambda = h/(mv)$ pour un électron ($m = 9.109 \times 10^{-31} \text{ kg}$) se déplaçant à $v = 10^6 \text{ m/s}$, avec $h = 6.626 \times 10^{-34} \text{ J}\cdot\text{s}$. Comparez avec la taille d'un atome ($\sim 10^{-10} \text{ m}$).

Résumé du chapitre

- `<MINTED>` affiche du texte et des valeurs.
- Python connaît les opérations : `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`.
- Les commentaires commencent par `<MINTED>`.
- Les erreurs sont normales — lisez le message de bas en haut.
- Un script `<MINTED>` est un programme réutilisable.

Chapitre 2

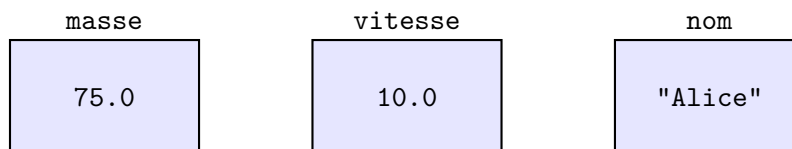
Variables, Types et Opérations de Base

Quand John Backus dirige chez IBM en 1957 le développement de Fortran — le premier langage de programmation de haut niveau largement utilisé — la promesse est simple : remplacer les instructions cryptiques en langage machine par des formules ressemblant aux mathématiques, comme $V = (4./3.) \times \text{PI} \times R^3$. Le mot *variable*, emprunté à l’algèbre, prend alors un sens nouveau en informatique : une étiquette nommée pour une case de mémoire, dont la valeur peut changer au cours de l’exécution. Soixante-cinq ans plus tard, ce concept est si naturel qu’on en oublie qu’il n’est pas évident : les langages fonctionnels purs (Haskell) ou les paradigmes immuables (Clojure, Rust) repensent la notion de variable. Ce chapitre construit le système de types de Python (entiers, flottants, chaînes, listes, dictionnaires, tuples, ensembles), explique le modèle d’objets sous-jacent (références, mutabilité, identité vs égalité), et présente les pièges courants (assignation vs copie, scope, type coercion).

2.1 Les variables : des boîtes avec des étiquettes

Intuition

Une **variable** est comme une boîte étiquetée dans laquelle on range une valeur. L’étiquette (le nom) permet de retrouver le contenu à tout moment.



Python — Créer des variables

```
<MINTED>
```

Sortie

```
Nom : Alice, Âge : 20  
Masse : 75.0 kg  
Étudiant : True
```

Attention

En Python, on ne « déclare » pas le type d’une variable. Python le déduit automatiquement. C’est le **typage dynamique**.

2.2 Les types de données

Définition 2.1 (Types fondamentaux). Python possède quatre types fondamentaux :

- **<MINTED>** — entier : **<MINTED>**, **<MINTED>**, **<MINTED>**
- **<MINTED>** — décimal : **<MINTED>**, **<MINTED>**, **<MINTED>**
- **<MINTED>** — chaîne : **<MINTED>**, **<MINTED>**
- **<MINTED>** — booléen : **<MINTED>**, **<MINTED>**

Python — Vérifier le type

<MINTED>

Sortie

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

2.3 Notation scientifique

Python — Notation scientifique

<MINTED>

Sortie

```
c  = 299800000.0
h  = 6.626e-34
Na = 6.022e+23
G  = 6.674e-11
```

2.4 Conversion de types

Python — Conversions

<MINTED>

Sortie

```
42.0 <class 'float'>
3 <class 'int'>
25 3.14
La réponse est 42
```

2.5 Opérations sur les chaînes

Python — Chaînes de caractères

<MINTED>

Sortie

```
Marie Curie
-----
Longueur : 11
Première lettre : M
Dernière lettre : e
MARIE CURIE
marie curie
Marie Skłodowska-Curie
```

index :

M	a	r	i	e		C	u	r	i	e
0	1	2	3	4	5	6	7	8	9	10

2.6 Les f-strings (chaînes formatées)

Python — f-strings

<MINTED>

Sortie

```
L'élément Hydrogène (Z=1) a une masse de 1.008 u
= 3.14
= 3.141593
= 3.14e+00
H      1.008 u
C      12.011 u
O      15.999 u
```

2.7 Opérateurs de comparaison et booléens

Python — Comparaisons

<MINTED>

Sortie

True
True
True
False
False
True
False

Opérateurs de comparaison

Opérateur	Signification	Exemple
<MINTED>	Égal à	<MINTED> → <MINTED>
<MINTED>	Différent de	<MINTED> → <MINTED>
<MINTED>, <MINTED>	Inférieur, supérieur	<MINTED> → <MINTED>
<MINTED>, <MINTED>	Inf. ou égal, sup. ou égal	<MINTED> → <MINTED>
<MINTED>	ET logique	<MINTED> → <MINTED>
<MINTED>	OU logique	<MINTED> → <MINTED>
<MINTED>	NON logique	<MINTED> → <MINTED>

2.8 L'entrée utilisateur

Python — input

<MINTED>

Sortie

Bonjour Alice, vous avez 20 ans.

2.9 Ce qui peut mal tourner

Ce qui peut mal tourner

Erreurs fréquentes

<MINTED>

2.10 Mini-projet : Fiche d'identité d'un atome

Mini-projet scientifique

Créez un programme qui affiche la fiche d'identité d'un atome.

Python

<MINTED>

Sortie

```
=====
FICHE D'IDENTITÉ : Carbone
=====
Symbole           : C
Numéro atomique   : 6
Masse atomique    : 12.011 u
Électronégativité : 2.55
Nombre de protons : 6
Nombre d'électrons : 6
=====
```

2.11 Exercices

Exercice 2.1 (★ — Guidé). Créez des variables pour stocker la constante de Boltzmann ($k_B = 1.381 \times 10^{-23}$ J/K) et une température $T = 300$ K. Calculez l'énergie thermique $E = k_B T$ et affichez le résultat.

Exercice 2.2 (★ — Guidé). Créez une variable <MINTED> (rayon de la Terre en km). Calculez et affichez la circonférence ($C = 2\pi r$) et la surface ($S = 4\pi r^2$) de la Terre.

Exercice 2.3 (★★ — Indépendant). Écrivez un programme qui demande à l'utilisateur une distance en kilomètres et la convertit en miles (1 km = 0.621 miles), en mètres et en centimètres.

Exercice 2.4 (★★ — Indépendant). Créez un programme qui calcule l'IMC (Indice de Masse Corporelle) : $IMC = \frac{\text{masse (kg)}}{\text{taille (m)}^2}$. Affichez le résultat avec 1 décimale.

Exercice 2.5 (★★★ — Défi scientifique). La formule de l'énergie d'un photon est $E = hf = hc/\lambda$. Calculez l'énergie d'un photon de lumière rouge ($\lambda = 700$ nm) et de lumière bleue ($\lambda = 450$ nm). Exprimez les résultats en joules et en électron-volts (1 eV = 1.602×10^{-19} J).

Résumé du chapitre

- Une variable stocke une valeur : <MINTED>.
- Types : <MINTED>, <MINTED>, <MINTED>, <MINTED>.
- <MINTED> donne le type, <MINTED>/<MINTED>/<MINTED> convertissent.

- Notation scientifique : `<MINTED>` = 6.022×10^{23} .
- f-strings : `<MINTED>` pour le formatage.
- Opérateurs de comparaison : `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`, `<MINTED>`.

Chapitre 3

Structures de Contrôle — Conditions et Boucles

Intuition

Imaginez une recette de cuisine : « Si la pâte est trop liquide, ajoutez de la farine ; sinon, enfournez pendant 30 minutes. » Un programme fonctionne exactement de la même manière : il prend des *décisions* et *répète* des étapes selon des conditions. Les structures de contrôle sont le cœur de tout algorithme scientifique.

3.1 Les conditions : <MINTED>, <MINTED>, <MINTED>

En science, on classe souvent selon des seuils. Considérons les *transitions de phase* de l'eau :

Python

```
<MINTED>
```

Sortie

À -5°C, l'eau est solide.

Remarque 3.1. Python évalue les conditions *dans l'ordre*. Dès qu'une condition est vraie, le bloc correspondant est exécuté et les suivants sont ignorés.

3.1.1 Opérateurs de comparaison et opérateurs logiques

Définition 3.2 (Opérateurs de comparaison). Les opérateurs <MINTED>, <MINTED>, <MINTED>, <MINTED>, <MINTED>, <MINTED> renvoient un booléen (<MINTED> ou <MINTED>). On peut combiner des conditions avec <MINTED>, <MINTED>, <MINTED>.

Python

```
<MINTED>
```

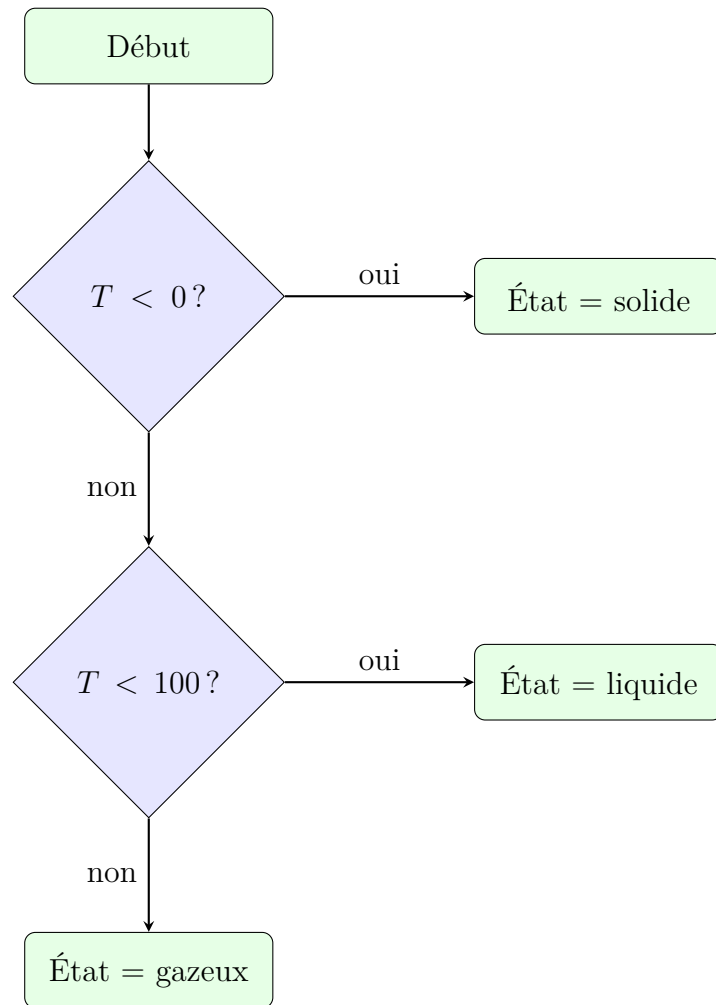


FIG. 3.1 : Organigramme d'une structure **<MINTED>** pour les transitions de phase.

Sortie

Solution acide

3.2 La boucle **<MINTED>**

La boucle **<MINTED>** répète un bloc *tant qu'une condition est vraie*. En sciences, on l'utilise souvent pour des algorithmes de *convergence*.

Exemple 3.3 (Convergence d'une suite). La suite $u_{n+1} = \frac{1}{2} \left(u_n + \frac{a}{u_n} \right)$ converge vers $\sqrt{a}$ (méthode de Héron).

Python

<MINTED>

Sortie

sqrt(2.0) 1.4142135623730951
Convergence en 4 itérations

Attention

Une boucle <MINTE> dont la condition ne devient *jamais* fausse tourne à l'infini !
Ajoutez toujours un garde-fou :
<MINTE>

3.3 La boucle <MINTE> et <MINTE>

La boucle <MINTE> parcourt un *itérable* (liste, chaîne, <MINTE>, etc.).

Python

<MINTE>

Sortie

Somme des carrés de 1 à 10 = 385
Formule : 385

Définition 3.4 (<MINTE>). <MINTE> génère les entiers de <MINTE> à <MINTE> avec un pas de <MINTE>. Attention : la borne supérieure <MINTE> est *exclue*.

Python

<MINTE>

Sortie

0 3 6 9 12 15

3.4 Boucles imbriquées

Python

<MINTE>

Sortie

$1 \times 1 = 1$	$1 \times 2 = 2$	$1 \times 3 = 3$
$2 \times 1 = 2$	$2 \times 2 = 4$	$2 \times 3 = 6$
$3 \times 1 = 3$	$3 \times 2 = 6$	$3 \times 3 = 9$

3.5 <MINTE> et <MINTE>

Python

<MINTE>

Sortie

Premier nombre premier > 50 : 53

Python

<MINTED>

Sortie

Moyenne (valeurs positives) = 4.43

3.6 Erreurs courantes

Ce qui peut mal tourner

1. **Erreur d'indentation** : Python utilise l'indentation pour délimiter les blocs. Mélanger espaces et tabulations provoque une <MINTED>. Règle : utilisez toujours **4 espaces**.
2. **Boucle infinie** : oublier d'incrémenter la variable de contrôle dans un <MINTED> mène à une boucle qui ne s'arrête jamais.
<MINTED>
3. **Erreur de décalage (*off-by-one*)** : <MINTED> produit les entiers de 1 à **9**, pas 10 ! Pour inclure 10, écrivez <MINTED>.
4. **Utiliser <MINTED> au lieu de <MINTED>** : <MINTED> est l'affectation, <MINTED> est la comparaison. Écrire <MINTED> provoque une <MINTED>.

3.7 Mini-projet : Désintégration radioactive

Mini-projet scientifique

La loi de désintégration radioactive donne le nombre d'atomes restants :

$$N(t) = N_0 \times 0,5^{t/t_{1/2}}$$

où N_0 est le nombre initial d'atomes et $t_{1/2}$ la demi-vie.

Objectif : simuler la désintégration du carbone-14 ($t_{1/2} = 5730$ ans) et afficher le nombre d'atomes restants chaque 1000 ans.

Python

<MINTED>

Sortie

Temps (ans)	N restants	% restant
0	1000000	100.0%
1000	886076	88.6%
2000	785128	78.5%
3000	695685	69.6%
4000	616441	61.6%
5000	546274	54.6%
...		
37000	12055	1.2%

Après 38000 ans, il reste moins de 1% des atomes.

3.8 Exercices

Exercice 3.1 (★ — Classification de l'IMC). Écrivez un programme qui demande le poids (kg) et la taille (m) d'une personne, calcule l'IMC ($\text{IMC} = \text{poids}/\text{taille}^2$) et affiche la catégorie : sous-poids ($< 18,5$), normal ($18,5\text{--}25$), surpoids ($25\text{--}30$), obésité (≥ 30).

Exercice 3.2 (★ — Compte à rebours). Utilisez une boucle **<MINTED>** pour afficher un compte à rebours de 10 à 0, puis affichez « Décollage! ».

Exercice 3.3 (★★ — Suite de Fibonacci). Calculez les 20 premiers termes de la suite de Fibonacci ($F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$) avec une boucle **<MINTED>**. Vérifiez que le rapport F_n/F_{n-1} converge vers le nombre d'or $\varphi \approx 1,618$.

Exercice 3.4 (★★ — Crible d'Ératosthène). Utilisez des boucles imbriquées pour trouver tous les nombres premiers inférieurs à 100.

Exercice 3.5 (★★★ — Simulation de marche aléatoire). Simulez une marche aléatoire 1D : à chaque pas, la particule se déplace de +1 ou -1 avec probabilité égale. Après $N = 1000$ pas, affichez la position finale. Répétez 100 fois et calculez la distance moyenne $\langle |x| \rangle$. Comparez avec la prédiction théorique $\sqrt{N} \approx 31,6$. *Indice* : utilisez **<MINTED>** et **<MINTED>**.

Fonctions clés

Résumé du chapitre : Structures de contrôle

- **Conditions** : **<MINTED>** / **<MINTED>** / **<MINTED>**
- **Boucle <MINTED>** : répète tant que la condition est vraie
- **Boucle <MINTED>** : parcourt un itérable (**<MINTED>**, liste, etc.)
- **<MINTED>** : entiers de **<MINTED>** à **<MINTED>**, pas de **<MINTED>**
- **<MINTED>** : sort immédiatement de la boucle
- **<MINTED>** : passe à l'itération suivante

- **Indentation** : 4 espaces, toujours cohérente
- **Garde-fou** : prévoir un nombre maximal d'itérations pour **<MINTED>**

Chapitre 4

Fonctions et Portée

Intuition

Une fonction est comme une *machine* dans un laboratoire : on y introduit des *entrées* (réactifs), elle effectue un *traitement* (réaction chimique) et produit une *sortie* (produit). Plutôt que de reconstruire la machine à chaque expérience, on la réutilise. C'est le principe **DRY** : « *Don't Repeat Yourself* ».

4.1 Pourquoi utiliser des fonctions ?

- **Réutilisabilité** : écrire une formule une seule fois et l'appeler partout.
- **Lisibilité** : un programme découpé en fonctions est plus facile à comprendre.
- **Débogage** : on peut tester chaque fonction indépendamment.
- **Modularité** : chaque fonction a une responsabilité unique.

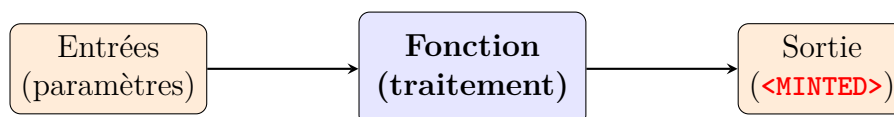


FIG. 4.1 : Une fonction vue comme une machine : entrées $\rightarrow$ traitement $\rightarrow$ sortie.

4.2 Définir et appeler une fonction

Définition 4.1 (Syntaxe d'une fonction). **<MINTED>**

Python

<MINTED>

Sortie

E = 9.0 J

4.3 Docstrings

Une *docstring* est une chaîne de documentation placée juste après `<MINTED>`. Elle est accessible via `<MINTED>`.

Python

`<MINTED>`

Sortie

Force Terre-Lune : 1.98e+20 N

4.4 Arguments par défaut et arguments nommés

Python

`<MINTED>`

Sortie

Terre : y = 19.62 m
Lune : y = 3.24 m
Avec v0=5 : y = 29.62 m

Bonne pratique

Utilisez des arguments nommés pour améliorer la lisibilité : `<MINTED>` est plus clair que `<MINTED>`.

4.5 Fonctions avec plusieurs valeurs de retour

Python

`<MINTED>`

Sortie

r = 5.00, = 53.1°

4.6 Portée des variables : locale vs globale

Définition 4.2 (Portée (*scope*)). Une variable définie à l'intérieur d'une fonction est **locale** : elle n'existe que pendant l'exécution de la fonction. Une variable définie en dehors est **globale**.

Python

`<MINTED>`

Sortie

Dans la fonction : $x = 5$
En dehors : $x = 10$

Attention

Évitez d'utiliser `<MINTED>` pour modifier des variables globales depuis une fonction. Préférez *toujours* passer les valeurs en paramètres et utiliser `<MINTED>`.

4.7 Fonctions lambda

Les fonctions `<MINTED>` sont des fonctions anonymes courtes, utiles pour des opérations simples.

Python`<MINTED>`**Sortie**

$-40^{\circ}\text{C} = 233.15 \text{ K}$
 $0^{\circ}\text{C} = 273.15 \text{ K}$
 $20^{\circ}\text{C} = 293.15 \text{ K}$
 $37^{\circ}\text{C} = 310.15 \text{ K}$
 $100^{\circ}\text{C} = 373.15 \text{ K}$

4.8 Erreurs courantes

Ce qui peut mal tourner

1. **Oublier `<MINTED>`** : sans `<MINTED>`, une fonction renvoie `<MINTED>` par défaut. Le calcul est effectué mais le résultat est perdu !
`<MINTED>`
2. **Argument mutable par défaut** : utiliser une liste comme valeur par défaut est dangereux, car elle est partagée entre tous les appels !
`<MINTED>`
3. **Confondre `<MINTED>` et `<MINTED>`** : `<MINTED>` affiche à l'écran mais ne renvoie pas de valeur utilisable dans une expression.

4.9 Mini-projet : Calculateur de formules physiques

Mini-projet scientifique

Créez un module contenant les fonctions suivantes :

- **<MINTED>** : $E_c = \frac{1}{2}mv^2$
- **<MINTED>** : $F = G\frac{m_1m_2}{r^2}$
- **<MINTED>** : $T = 2\pi\sqrt{L/g}$
- **<MINTED>** : $V = RI$ (donner deux grandeurs, calculer la troisième)

Chaque fonction doit avoir une docstring complète. Testez-les avec des valeurs connues.

Python

<MINTED>

Sortie

Pendule L=1m : T = 2.006 s
 Loi d'Ohm : V = 50.0 V
 Loi d'Ohm : I = 0.120 A

4.10 Exercices

Exercice 4.1 (★ — Conversion de températures). Écrivez deux fonctions **<MINTED>** et **<MINTED>**. Testez avec l'eau bouillante ($100^\circ\text{C} = 212^\circ\text{F}$).

Exercice 4.2 (★ — Factorielle). Écrivez une fonction **<MINTED>** qui calcule $n!$ avec une boucle. Vérifiez que **<MINTED>**.

Exercice 4.3 (★★ — Fonction récursive). Réécrivez la factorielle de manière *récursive* : une fonction qui s'appelle elle-même. Quel est le cas de base ?

Exercice 4.4 (★★ — Statistiques descriptives). Écrivez une fonction **<MINTED>** qui reçoit une liste de nombres et renvoie la moyenne, la médiane et l'écart-type (sans utiliser de bibliothèque).

Exercice 4.5 (★★★ — Méthode de Newton). Implémentez la méthode de Newton pour trouver les zéros d'une fonction : $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$. Votre fonction prendra en paramètres **<MINTED>**, **<MINTED>** (dérivée), **<MINTED>** (estimation initiale) et **<MINTED>** (tolérance). Testez avec $f(x) = x^2 - 2$ pour retrouver $\sqrt{2}$.

Fonctions clés

Résumé du chapitre : Fonctions et portée

- Définition : **<MINTED>** ... **<MINTED>**

- **Docstring** : chaîne de documentation entre triple guillemets
- **Arguments par défaut** : **<MINTED>** — n vaut 10 si non précisé
- **Arguments nommés** : **<MINTED>** pour plus de clarté
- **Valeurs multiples** : **<MINTED>** renvoie un tuple
- **Portée locale** : les variables dans une fonction sont isolées
- **Lambda** : **<MINTED>** pour des fonctions courtes
- **Principe DRY** : ne jamais copier-coller du code, écrire une fonction

Chapitre 5

Structures de Données

Intuition

Un scientifique ne travaille jamais avec une seule donnée : il manipule des séries de mesures, des tableaux de résultats, des catalogues d'espèces. Python offre plusieurs *structures de données* pour organiser ces informations — chacune avec ses forces, comme les différents contenants d'un laboratoire (éprouvettes, boîtes de Pétri, classeurs).

5.1 Les listes

Définition 5.1 (Liste). Une **liste** est une collection *ordonnée* et *modifiable* d'éléments. On la crée avec des crochets : **<MINTED>**.

5.1.1 Création et indexation

Python

<MINTED>

Sortie

Premier jour : 18.2°C
Dernier jour : 19.7°C
Nombre de mesures : 7

5.1.2 Slicing (découpage)

Python

<MINTED>

Sortie

Jours ouvrés : [18.2, 19.5, 17.8, 21.0, 22.3]
Un sur deux : [18.2, 17.8, 22.3, 19.7]
Inversé : [19.7, 20.1, 22.3, 21.0, 17.8, 19.5, 18.2]

5.1.3 Méthodes utiles

Python

<MINTED>

Sortie

Après append : [12.5, 8.3, 15.7, 10.2, 9.8]
Retiré : 9.8, liste : [12.5, 8.3, 15.7, 10.2]
Trié : [8.3, 10.2, 12.5, 15.7]
Inversé : [15.7, 12.5, 10.2, 8.3]

5.2 Compréhensions de listes

Les compréhensions offrent une syntaxe compacte pour créer des listes.

Python

<MINTED>

Sortie

Carrés : [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Jours chauds : [21.0, 22.3, 20.1]
Fahrenheit : ['64.8', '67.1', '64.0', '69.8', '72.1', '68.2', '67.5']

5.3 Les tuples

Définition 5.2 (Tuple). Un **tuple** est une collection *ordonnée* mais *immuable* (non modifiable). On le crée avec des parenthèses : <MINTED>.

Python

<MINTED>

Sortie

vitesse lumière = 299792458 m/s
vitesse lumière : 299792458 m/s

Remarque 5.3. Utilisez les tuples pour des données qui ne doivent pas être modifiées : coordonnées (x, y) , constantes physiques, clés de dictionnaire.

5.4 Les dictionnaires

Définition 5.4 (Dictionnaire). Un **dictionnaire** associe des *clés* à des *valeurs*. On le crée avec des accolades : <MINTED>.

5.4.1 Le tableau périodique comme dictionnaire

Python

<MINTED>

Sortie

Carbone : Z=6, masse=12.011 u
Masse molaire H O = 18.015 g/mol

5.4.2 Itération sur un dictionnaire

Python

<MINTED>

Sortie

H : Hydrogène Z= 1
He : Hélium Z= 2
C : Carbone Z= 6
N : Azote Z= 7
O : Oxygène Z= 8
Fe : Fer Z=26

5.5 Les ensembles

Python

<MINTED>

Sortie

Communs : {'O', 'H', 'C'}
Tous : {'S', 'O', 'Fe', 'H', 'Ca', 'N', 'C'}
Unique à A : {'S', 'N'}

Liste	Tuple	Dictionnaire	Ensemble
Ordonnée	Ordonné	Clé → Valeur	Non ordonné
Modifiable	Immuable	Modifiable	Pas de doublons
<MINTED>	<MINTED>	<MINTED>	<MINTED>

FIG. 5.1 : Comparaison des structures de données en Python.

5.6 Erreurs courantes

Ce qui peut mal tourner

1. **Index hors limites** (**<MINTED>**) : une liste de 5 éléments a des indices de 0 à 4. Accéder à l'indice 5 provoque une erreur.
<MINTED>
2. **Modifier une liste pendant l'itération** : supprimer des éléments d'une liste en la parcourant produit des résultats imprévisibles.
<MINTED>
3. **Clé inexistante** (**<MINTED>**) : accéder à une clé absente d'un dictionnaire. Utilisez **<MINTED>** pour éviter l'erreur.
4. **Confondre liste et tuple** : tenter de modifier un tuple avec **<MINTED>** provoque un **<MINTED>**.

5.7 Mini-projet : Gestion des notes étudiantes

Mini-projet scientifique

Créez un système de gestion de notes utilisant un dictionnaire :

- Clés : noms des étudiants
- Valeurs : listes de notes
- Fonctions : ajouter un étudiant, ajouter une note, calculer la moyenne, trouver le major de promotion

Python

<MINTED>

Sortie

Étudiant	Notes	Moyenne
Alice	15.5, 17.0, 14.0	15.50
Bob	12.0, 11.5, 13.0	12.17
Claire	18.0, 16.5, 19.0	17.83

Major de promotion : Claire (17.83/20)

5.8 Exercices

Exercice 5.1 (★ — Manipulation de listes). Créez une liste de 10 mesures de pH. Calculez la moyenne, le minimum et le maximum *sans utiliser* les fonctions `<MINTED>`, `<MINTED>`, `<MINTED>`.

Exercice 5.2 (★ — Filtrage). À partir d'une liste de températures, créez avec une compréhension de liste une nouvelle liste ne contenant que les valeurs entre 15 °C et 25 °C.

Exercice 5.3 (★★ — Compteur de fréquences). Écrivez une fonction qui reçoit une chaîne de caractères et renvoie un dictionnaire comptant la fréquence de chaque lettre. Testez avec une séquence ADN (« ATCGGATCCA »).

Exercice 5.4 (★★ — Matrice comme liste de listes). Représentez une matrice 3×3 comme une liste de listes. Écrivez une fonction qui calcule la transposée. Vérifiez avec la matrice identité.

Exercice 5.5 (★★★ — Analyse de séquence protéique). Créez un dictionnaire associant chaque acide aminé à sa masse molaire. Écrivez une fonction qui, étant donnée une séquence d'acides aminés (lettres), calcule la masse totale de la protéine. Testez avec « MKVLWA ».

Fonctions clés

Résumé du chapitre : Structures de données

- **Liste** `<MINTED>` : ordonnée, modifiable, méthodes `<MINTED>`, `<MINTED>`, `<MINTED>`
- **Compréhension** : `<MINTED>`
- **Tuple** `<MINTED>` : ordonné, immuable, idéal pour les constantes
- **Dictionnaire** `<MINTED>` : accès par clé, méthodes `<MINTED>`, `<MINTED>`, `<MINTED>`
- **Ensemble** `<MINTED>` : pas de doublons, opérations $\cap$, $\cup$, $\setminus$
- **Indexation** : commence à 0, indices négatifs depuis la fin
- **Slicing** : `<MINTED>`

Chapitre 6

NumPy — Tableaux et Calcul Numérique

Intuition

Imaginez devoir mesurer la température en 1 000 points d'une plaque chauffante. Avec une liste Python, chaque opération nécessite une boucle. NumPy permet de traiter *tous les points en une seule instruction*, comme un instrument qui mesure toute la plaque d'un coup. C'est la différence entre compter des grains de sable un par un et peser le tas entier.

6.1 Pourquoi NumPy ?

Python

<MINTED>

Sortie

Liste Python : 0.2851 s
NumPy : 0.0032 s
Accélération : ×89

6.2 Création de tableaux

Python

<MINTED>

Sortie

Array : [1. 2. 3. 4.]
Zéros : [0. 0. 0. 0. 0.]
Uns : [1. 1. 1. 1.]
Arange : [0 2 4 6 8]

```
Linspace : [0.  0.25 0.5  0.75 1.  ]
```

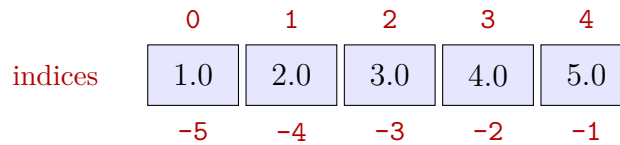


FIG. 6.1 : Indexation d'un tableau NumPy : indices positifs (haut) et négatifs (bas).

6.3 Opérations élément par élément

Python

<MINTED>

Sortie

```
a + b = [11 22 33 44]
a * b = [ 10  40  90 160]
a ** 2 = [ 1  4  9 16]
a + 100 = [101 102 103 104]
a > 2 = [False False  True  True]
```

Définition 6.1 (Broadcasting). Le **broadcasting** permet à NumPy d'appliquer des opérations entre tableaux de tailles différentes. Un scalaire est automatiquement « étendu » à la taille du tableau.

6.4 Indexation et slicing

Python

<MINTED>

Sortie

```
Élément 3 : 40
Slice [2:5] : [30 40 50]
Masque : [False False False  True  True  True  True]
Valeurs > 35 : [40 50 60 70]
Après modif : [ 0  0 30 40 50 60 70]
```

6.5 Fonctions mathématiques

Python

<MINTED>

Sortie

```
x          = [0.   1.57 3.14 4.71 6.28]
sin(x)     = [ 0.    1.    0.   -1.   -0.    ]
cos(x)     = [ 1.    0.   -1.   -0.    1.    ]
exp(x)     = [ 1.    4.81 23.14 111.32 535.49]

t          = [0.  1.  2.  3.  4.  5.]
signal     = [ 0.   -0.    0.   -0.    0.   -0.    ]
```

6.6 Statistiques descriptives

Python

<MINTED>

Sortie

```
Moyenne      : 12.09 mg/L
Écart-type   : 0.27 mg/L
Minimum      : 11.70 mg/L
Maximum      : 12.50 mg/L
Médiane      : 12.05 mg/L
```

6.7 Algèbre linéaire élémentaire

Python

<MINTED>

Sortie

```
A @ B =
[[19 22]
 [43 50]]

det(A) = -2.0
A-1 =
[[-2.   1. ]
 [ 1.5 -0.5]]
```

6.8 Nombres aléatoires

Python

<MINTED>

Sortie

```
Uniforme : [0.773 0.438 0.859 0.697 0.094]
Normale  : [ 0.314  0.459 -0.537  0.268 -0.232]
Lancers  : [5 2 3 6 1 4 2 5 3 6]
```

6.9 Erreurs courantes

Ce qui peut mal tourner

1. **Incompatibilité de dimensions** (<MINTED>) : opérer sur des tableaux de tailles incompatibles provoque une <MINTED>.
2. **Oublier les opérations élément par élément** : l'opérateur <MINTED> entre deux tableaux fait une multiplication élément par élément, *pas* un produit matriciel. Utilisez <MINTED> ou <MINTED> pour le produit matriciel.
3. **Confusion** <MINTED> / <MINTED> : <MINTED> spécifie un *pas*, <MINTED> spécifie un *nombre de points*.
4. **Modifier une vue au lieu d'une copie** : le slicing NumPy crée une *vue* (pas une copie). Modifier la vue modifie l'original!

<MINTED>

6.10 Mini-projet : Estimation de π par Monte Carlo

Mini-projet scientifique

La méthode de Monte Carlo estime π en tirant des points aléatoires dans un carré $[-1, 1]^2$ et en comptant ceux qui tombent dans le cercle unité ($x^2 + y^2 \leq 1$). Le rapport donne :

$$\pi \approx 4 \times \frac{\text{points dans le cercle}}{\text{points totaux}}$$

Python

<MINTED>

Sortie

```
Points totaux      : 1,000,000
Points dans cercle : 785,436
  estimé          : 3.141744
  exact           : 3.141593
Erreur            : 0.000151
```

6.11 Exercices

Exercice 6.1 (★ — Création de tableaux). Créez les tableaux suivants : (a) les entiers de 0 à 99, (b) 50 valeurs équidistantes entre $-\pi$ et π , (c) une matrice 3×3 de zéros avec des 1 sur la diagonale (matrice identité).

Exercice 6.2 (★ — Conversion de températures). Créez un tableau de températures en Celsius de -40 à 100 (pas de 10). Convertissez-le en Fahrenheit ($F = 1,8 \times C + 32$) *sans boucle*.

Exercice 6.3 (★★ — Analyse de données expérimentales). Générez 1000 mesures simulées d'une distribution normale ($\mu = 50$, $\sigma = 5$). Calculez la moyenne, l'écart-type, et comptez combien de mesures sont à plus de 2σ de la moyenne. Comparez avec la prédiction théorique ($\approx 5\%$).

Exercice 6.4 (★★ — Produit scalaire). Calculez l'angle entre les vecteurs $\vec{u} = (1, 2, 3)$ et $\vec{v} = (4, 5, 6)$ en utilisant la formule $\cos \theta = \frac{\vec{u} \cdot \vec{v}}{|\vec{u}| |\vec{v}|}$.

Exercice 6.5 (★★★ — Résolution de système linéaire). Résolvez le système d'équations linéaires suivant avec **<MINTED>** :

$$\begin{cases} 2x + 3y - z = 1 \\ 4x + y + 2z = 2 \\ -x + 2y + 3z = 3 \end{cases}$$

Vérifiez votre résultat en calculant $A\vec{x}$ et en comparant avec $\vec{b}$.

Fonctions clés

Résumé du chapitre : NumPy

- **Création** : **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**
- **Opérations** : **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>** s'appliquent élément par élément
- **Broadcasting** : un scalaire est étendu automatiquement
- **Maths** : **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**
- **Stats** : **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**, **<MINTED>**
- **Algèbre** : **<MINTED>** (produit matriciel), **<MINTED>**, **<MINTED>**, **<MINTED>**

- Aléatoire : **<MINTED>**
- Indexation booléenne : **<MINTED>** filtre les éléments positifs

Chapitre 7

Matplotlib — Visualisation Scientifique

Intuition

« Une image vaut mille mots », et en science, un graphique vaut mille tableaux de données. Matplotlib est la bibliothèque de référence en Python pour créer des figures de qualité publication. Pensez-y comme le « papier millimétré numérique » du scientifique moderne.

7.1 Premier graphique : `<MINTED>`

Python

```
<MINTED>
```

Sortie

[Figure affichée : deux courbes $\sin(x)$ et $\cos(x)$ sur $[0, 2]$, avec légende, grille et titres des axes.]

Remarque 7.1. La convention est d'importer `<MINTED>`. Toutes les fonctions de traçage sont alors accessibles via `<MINTED>`.

7.2 Nuages de points : `<MINTED>`

Python

```
<MINTED>
```

Sortie

[Figure affichée : nuage de points bleus avec droite de régression rouge, titre « Loi de Beer-Lambert », axes étiquetés.]

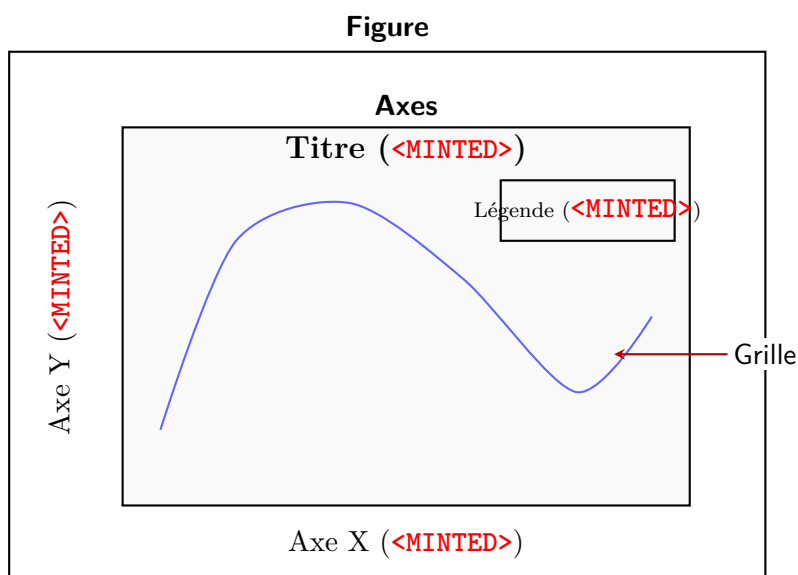


FIG. 7.1 : Anatomie d'une figure Matplotlib : figure, axes, titre, étiquettes et légende.

7.3 Histogrammes : <MINTED>

Python

<MINTED>

Sortie

[Figure affichée : histogramme vert avec 30 barres, ligne rouge en pointillés marquant la moyenne.]

7.4 Diagrammes en barres : <MINTED>

Python

<MINTED>

Sortie

[Figure affichée : diagramme en barres colorées montrant l'abondance des éléments dans le corps humain.]

7.5 Sous-graphiques : <MINTED>

Python

<MINTED>

Sortie

[Figure affichée : grille 2×2 de sous-graphiques avec $\sin(x)$, $\cos(x)$, signal amorti et polynômes.]

Bonne pratique

Utilisez toujours `<MINTED>` ou `<MINTED>` pour éviter que les étiquettes se chevauchent entre sous-graphiques.

7.6 Personnalisation avancée

Python

```
<MINTED>
```

Sortie

[Figure affichée : deux signaux sinusoïdaux avec styles différents, annotation fléchée pointant vers le maximum.]

7.7 Sauvegarder une figure : `<MINTED>`

Python

```
<MINTED>
```

Sortie

Figure sauvegardée en PNG et PDF.

Attention

Appelez `<MINTED>` avant `<MINTED>`, sinon la figure sera vide lors de la sauvegarde. L'option `<MINTED>` évite de couper les étiquettes.

7.8 Figures scientifiques : trajectoire d'un projectile

Python

```
<MINTED>
```

Sortie

[Figure affichée : quatre trajectoires paraboliques pour différents angles de lancement, montrant que 45° donne la portée maximale.]

7.9 Erreurs courantes

Ce qui peut mal tourner

1. **Oublier `<MINTED>`** : dans un script (hors Jupyter), sans `<MINTED>`, aucune fenêtre ne s'affiche. La figure est créée en mémoire mais reste invisible.
2. **Mauvaise taille de figure** : par défaut, les figures sont petites. Utilisez `<MINTED>` pour contrôler la taille en pouces.
3. **Sauvegarder après `<MINTED>`** : `<MINTED>` vide la figure en cours. Tout appel à `<MINTED>` après donnera un fichier vide.
4. **Confondre `<MINTED>` et l'interface orientée objet** : pour des sous-graphiques, utilisez `<MINTED>` et non `<MINTED>` qui agit sur le dernier axe actif.
5. **Oublier les étiquettes** : un graphique sans titre ni étiquettes d'axes est inutilisable en contexte scientifique. Ajoutez *toujours* `<MINTED>`, `<MINTED>` et `<MINTED>`.

7.10 Mini-projet : Visualisation du mouvement d'un projectile

Mini-projet scientifique

Créez un programme complet qui :

1. Demande la vitesse initiale v_0 et l'angle de lancement θ .
2. Calcule la trajectoire, la portée et la hauteur maximale.
3. Produit une figure avec :
 - La trajectoire parabolique.
 - Un point marqué au sommet (hauteur max).
 - Une annotation indiquant la portée.
 - Un encadré avec les paramètres (v_0 , θ , portée, $h_{\max}$).
4. Sauvegarde la figure en PDF.

Python

```
<MINTED>
```

Sortie

Portée : 59.2 m, Hauteur max : 21.4 m

7.11 Exercices

Exercice 7.1 (★ — Courbe exponentielle). Tracez la fonction $f(x) = e^{-x/3} \cos(2\pi x)$ sur $[0, 10]$. Ajoutez un titre, des étiquettes d'axes et une grille.

Exercice 7.2 (★ — Histogramme de lancers de dés). Simulez 10 000 lancers de deux dés. Tracez l'histogramme de la somme obtenue. Quelle somme est la plus fréquente ?

Exercice 7.3 (★★ — Diagramme de phases). Tracez un diagramme de phases simplifié de l'eau : pression (y) en fonction de la température (x). Utilisez `<MINTED>` pour colorier les zones solide, liquide et gazeuse.

Exercice 7.4 (★★ — Sous-graphiques multiples). Créez une figure 2×2 montrant : (a) $\sin(x)$, (b) $\cos(x)$, (c) $\tan(x)$ (limité à $[-5, 5]$), (d) e^x et $\ln(x)$. Chaque sous-graphique doit avoir son titre et sa grille.

Exercice 7.5 (★★★ — Carte de chaleur). Créez une matrice 20×20 représentant la température d'une plaque chauffante (bords froids à 0°C , centre chaud à 100°C). Affichez-la avec `<MINTED>` et ajoutez une barre de couleur. *Indice* : initialisez une matrice de zéros et ajoutez une gaussienne centrée.

Fonctions clés

Résumé du chapitre : Matplotlib

- Courbes : `<MINTED>`
- Nuages : `<MINTED>`
- Histogrammes : `<MINTED>`
- Barres : `<MINTED>`
- Sous-graphiques : `<MINTED>`
- Étiquettes : `<MINTED>`, `<MINTED>`, `<MINTED>`
- Légende : `<MINTED>` (après avoir utilisé `<MINTED>`)
- Grille : `<MINTED>`
- Sauvegarde : `<MINTED>`
- Règle d'or : `<MINTED>` avant `<MINTED>`

Chapitre 8

Pandas — Analyse de Données

Intuition

Imaginez un tableur comme Excel, mais piloté entièrement par du code Python. C'est exactement ce que propose **Pandas** : une bibliothèque puissante pour charger, explorer, filtrer et analyser des données tabulaires. En sciences, la majorité des données expérimentales se présentent sous forme de tableaux — Pandas est l'outil idéal pour les manipuler.

8.1 Séries et DataFrames

Pandas repose sur deux structures fondamentales :

Définition 8.1 (Series et DataFrame). Une **Series** est une colonne de données étiquetée (comme un vecteur avec un index). Un **DataFrame** est un tableau à deux dimensions composé de plusieurs Series partageant le même index.

DataFrame

index	nom	masse	charge
0	proton	1.673	+1
1	neutron	1.675	0
2	électron	0.0009	−1

Index Colonnes (Series)

Python

```
<MINTED>
```

Sortie

```
proton      1.6730
neutron     1.6750
electron    0.0009
dtype: float64
```

8.2 Charger des données : <MINTED>

En pratique, on charge rarement des données à la main. Pandas lit de nombreux formats : CSV, Excel, JSON, SQL, etc. Nous utiliserons le jeu de données `tips` disponible dans Seaborn.

Python

<MINTED>

Sortie

```
<class 'pandas.core.frame.DataFrame'>
(244, 7)
```

Le DataFrame <MINTED> contient 244 lignes (observations) et 7 colonnes.

8.3 Exploration rapide

Python

<MINTED>

Sortie

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3

Python

<MINTED>

Sortie

	total_bill	tip	size
count	244.00	244.00	244.00
mean	19.79	3.00	2.57
std	8.90	1.38	0.95
min	3.07	1.00	1.00
25%	13.35	2.00	2.00
50%	17.80	2.90	2.00
75%	24.13	3.56	3.00
max	50.81	10.00	6.00

Python

<MINTED>

Sortie

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex        244 non-null    category
3   smoker     244 non-null    category
4   day        244 non-null    category
5   time       244 non-null    category
6   size       244 non-null    int64
```

8.4 Sélection de données

Définition 8.2 (loc vs iloc). **<MINTED>** sélectionne par **étiquettes** (noms de lignes/colonnes). **<MINTED>** sélectionne par **positions entières** (indices numériques).

Python

<MINTED>

Sortie

```
0    16.99
1    10.34
2    21.01
Name: total_bill, dtype: float64
1.01
1.01
   total_bill  tip
0      16.99  1.01
1      10.34  1.66
2      21.01  3.50
```

8.5 Filtrage avec des masques booléens

Python

<MINTED>

Sortie

```
   total_bill  tip  size
11      35.26  5.00    4
23      39.42  7.58    4
39      31.27  5.00    3
```

47	32.40	6.00	4
56	38.01	3.00	1
59	48.27	6.73	4
170	50.81	10.00	3
182	45.35	3.50	3
197	43.11	5.00	4
212	48.33	9.00	4

Python

<MINTED>

Sortie

```

Nombre de resultats : 11
      total_bill  tip  day
23          39.42  7.58  Sun
39          31.27  5.00  Sun
44          30.40  5.60  Sun
47          32.40  6.00  Sun
52          34.81  5.20  Sun

```

8.6 Agrégation avec GroupBy

L'opération <MINTED> permet de regrouper les données selon une variable catégorielle, puis d'appliquer une fonction d'agrégation.

Python

<MINTED>

Sortie

```

day
Thur    2.77
Fri     2.73
Sat     2.99
Sun     3.26
Name: tip, dtype: float64

```

Python

<MINTED>

Sortie

```

      total_bill      tip
sex  mean  std mean  std

```


Male	20.74	9.25	3.09	1.49
Female	18.06	8.01	2.83	1.16

8.7 Visualisation rapide avec <MINTED>

Pandas intègre matplotlib pour créer des graphiques en une ligne.

Python

<MINTED>

Python

<MINTED>

Remarque 8.3. Pour des visualisations plus élaborées, on peut utiliser Seaborn ou Matplotlib directement. La méthode <MINTED> est idéale pour une exploration rapide.

8.8 Créer de nouvelles colonnes

Python

<MINTED>

Sortie

	total_bill	tip	tip_pct
0	16.99	1.01	5.9
1	10.34	1.66	16.1
2	21.01	3.50	16.7
3	23.68	3.31	14.0

Ce qui peut mal tourner

Erreurs fréquentes avec Pandas

1. **SettingWithCopyWarning** — Quand on modifie une « tranche » d'un DataFrame, Pandas ne sait pas si on modifie l'original ou une copie. Solution : utilisez <MINTED> ou <MINTED> explicitement.

<MINTED>

2. **Confondre <MINTED> et <MINTED>** — <MINTED> accède à la ligne dont l'étiquette est 1 ; <MINTED> accède à la **deuxième** ligne (position 1). Après un filtrage, les étiquettes ne sont plus consécutives !

3. **Oublier les parenthèses dans les conditions multiples** — Écrivez <MINTED>, pas <MINTED>.

Mini-projet scientifiqueAnalyse du dataset `Tips`

1. Chargez le dataset `tips` avec Seaborn.
2. Calculez le pourboire moyen selon le moment du repas (`<MINTED>`).
3. Déterminez quel jour a le plus grand nombre de clients (somme de `<MINTED>`).
4. Créez une colonne `<MINTED> = <MINTED>`.
5. Tracez un histogramme de `<MINTED>` et commentez la distribution.
6. Testez si les fumeurs laissent un pourboire différent (`groupby + mean`).

8.9 Exercices

Exercice 8.1 (★). Chargez le dataset `<MINTED>`. Affichez les 5 premières lignes, la forme du DataFrame et le résumé statistique. Combien y a-t-il de valeurs manquantes par colonne ? (Indice : `<MINTED>`)

Exercice 8.2 (★). À partir du dataset `tips`, sélectionnez toutes les lignes où l'addition dépasse 30 € et le pourboire est inférieur à 3 €. Combien de lignes obtient-on ?

Exercice 8.3 (★★). Utilisez `<MINTED>` pour calculer la moyenne et l'écart-type du pourboire selon le sexe *et* le moment du repas (`groupby` sur deux colonnes). Quelle catégorie laisse le pourboire moyen le plus élevé ?

Exercice 8.4 (★★). Créez une nouvelle colonne `<MINTED>` qui vaut `<MINTED>` si `<MINTED>`, `<MINTED>` si entre 10 et 20, et `<MINTED>` sinon. Utilisez `<MINTED>` ou `<MINTED>`. Tracez un diagramme en barres du nombre de repas par catégorie.

Exercice 8.5 (★★★). Chargez le dataset `<MINTED>`. Analysez la distribution des méthodes de découverte d'exoplanètes au fil des années : groupez par `<MINTED>` et `<MINTED>`, comptez les découvertes, et tracez l'évolution temporelle pour les 3 méthodes les plus fréquentes.

Fonctions clés		
Chapitre 8 — Pandas : l'essentiel	Opération	Syntaxe
	Charger un CSV	<MINTED>
	Premières lignes	<MINTED>
	Dimensions	<MINTED>
	Résumé statistique	<MINTED>
	Sélection colonne	<MINTED> ou <MINTED>
	Sélection par position	<MINTED>
	Sélection par étiquette	<MINTED>
	Filtrage	<MINTED>
	Agrégation	<MINTED>
	Nouvelle colonne	<MINTED>
	Graphique rapide	<MINTED>

Chapitre 9

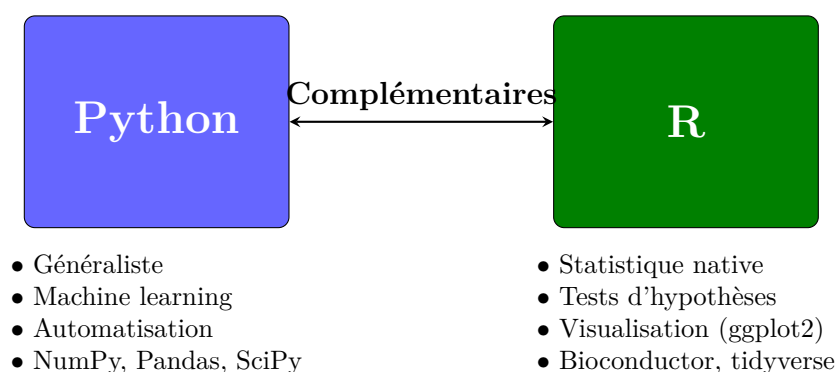
Introduction à R pour la Statistique

Intuition

Si Python est un couteau suisse généraliste, **R** est un instrument de précision conçu spécifiquement pour la statistique. Créé par des statisticiens pour des statisticiens, R excelle dans l'analyse de données, les tests d'hypothèses et la visualisation. De nombreux chercheurs en biologie, écologie et sciences sociales l'utilisent quotidiennement.

9.1 Pourquoi R ?

Définition 9.1 (Le langage R). **R** est un langage de programmation libre et gratuit, spécialisé dans le calcul statistique et la représentation graphique. Il dispose d'un écosystème de plus de 20 000 paquets sur le dépôt CRAN.



9.2 Variables et vecteurs

En R, l'opérateur d'affectation usuel est `<MINTED>` (flèche). La structure de base est le **vecteur**, créé avec la fonction `<MINTED>`.

R

`<MINTED>`

Sortie

```
[1] 12.3 14.1 11.8 13.5 12.9
[1] 1230 1410 1180 1350 1290
```

Attention

En R, l'indexation commence à **1** (et non 0 comme en Python). Ainsi, `<MINTED>` renvoie le *premier* élément, soit 12.3.

R

```
<MINTED>
```

Sortie

```
[1] 12.3
[1] 14.1 11.8 13.5
[1] 1 2 3 4 5 6 7 8 9 10
[1] 0 0 0 0 0
```

9.3 Data frames en R

R

```
<MINTED>
```

Sortie

```
      sujet dose effet
1         A   10   2.1
2         B   20   4.5
3         C   30   5.8
4         D   40   7.2
5         E   50   8.9
'data.frame': 5 obs. of  3 variables:
 $ sujet: chr  "A" "B" "C" "D" ...
 $ dose : num  10 20 30 40 50
 $ effet: num  2.1 4.5 5.8 7.2 8.9
```

9.4 Fonctions statistiques de base

R intègre nativement toutes les fonctions statistiques essentielles.

R

```
<MINTED>
```

Sortie

```
Moyenne      : 12.5
Mediane      : 12.5
Ecart-type   : 3.027650
Variance     : 9.166667
Min / Max    : 8 / 17
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  8.00  10.25   12.50   12.50   14.75   17.00
```

9.5 Tests statistiques

Définition 9.2 (Test de Student). Le **test t de Student** compare les moyennes de deux groupes pour déterminer si la différence observée est statistiquement significative ($p\text{-valeur} < 0.05$).

R

<MINTED>

Sortie

```
Welch Two Sample t-test

data:  traitement and controle
t = 5.1893, df = 9.5182, p-value = 0.0004507
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 2.633574 6.566426
sample estimates:
mean of x mean of y
 24.81667  20.21667
```

R

<MINTED>

Sortie

```
Pearson's product-moment correlation

data:  x and y
t = 42.085, df = 6, p-value = 4.386e-08
95 percent confidence interval:
 0.9971346 0.9999227
sample estimates:
      cor
0.9984112
```

9.6 Régression linéaire avec `<MINTED>`

R

`<MINTED>`

Sortie

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.14000	0.34351	0.408	0.71062
dose	0.17200	0.01033	16.649	0.00048 ***

Residual standard error: 0.3742 on 3 degrees of freedom

Multiple R-squared: 0.9893, Adjusted R-squared: 0.9857

Le coefficient $R^2 = 0.989$ indique un excellent ajustement linéaire : l'effet augmente de 0.172 unités par unité de dose.

9.7 Graphiques de base en R

R

`<MINTED>`

9.8 Comparaison Python vs R

Exemple 9.3 (Même analyse dans les deux langages).

Python

`<MINTED>`

R

`<MINTED>`

Remarque 9.4. Notez que `<MINTED>` en R calcule l'écart-type *corrigé* ($n - 1$) par défaut, comme `<MINTED>` en Python. La fonction `<MINTED>` sans `<MINTED>` divise par n , ce qui donne un résultat différent.

Ce qui peut mal tourner

Erreurs fréquentes en R

1. **Indexation à partir de 1** — En R, `<MINTED>` est le *premier* élément. En Python, `<MINTED>` est le *deuxième*. Cette différence est source de nombreuses erreurs quand on passe d'un langage à l'autre.
2. **`<MINTED>` vs `<MINTED>`** — En R, on privilégie `<MINTED>` pour l'affectation. Le signe `<MINTED>` fonctionne aussi dans la plupart des cas, mais `<MINTED>` est la

convention standard. Attention à ne pas écrire `<MINTED>` (comparaison !) au lieu de `<MINTED>` (affectation).

3. **Facteurs inattendus** — R convertit parfois automatiquement les chaînes en `factor`. Utilisez `<MINTED>` dans `<MINTED>` si nécessaire.
4. **Oublier `<MINTED>`** — Les fonctions comme `<MINTED>` retournent `<MINTED>` si le vecteur contient des valeurs manquantes. Ajoutez `<MINTED>`.

Mini-projet scientifique

Analyse statistique comparative en R Réalisez une étude complète en R :

1. Créez un data frame avec deux groupes expérimentaux (15 mesures chacun), simulés avec `<MINTED>`.
2. Calculez les statistiques descriptives de chaque groupe (`<MINTED>`, `<MINTED>`, `<MINTED>`).
3. Réalisez un test t de Student pour comparer les deux groupes.
4. Tracez des boîtes à moustaches côte à côte.
5. Ajoutez une troisième variable et effectuez une régression linéaire avec `<MINTED>`.
6. Interprétez les résultats : la différence est-elle significative ? Le modèle linéaire est-il pertinent (R^2) ?

9.9 Exercices

Exercice 9.1 (★). Créez un vecteur contenant les températures moyennes mensuelles d'une ville (12 valeurs). Calculez la moyenne, la médiane et l'écart-type. Quel mois est le plus chaud ? Utilisez `<MINTED>`.

Exercice 9.2 (★). Créez un data frame avec les colonnes `<MINTED>`, `<MINTED>` et `<MINTED>` pour 5 éléments chimiques. Affichez le résumé avec `<MINTED>` et accédez à la colonne `<MINTED>` avec `<MINTED>`.

Exercice 9.3 (★★). Générez 100 valeurs aléatoires suivant une loi normale ($\mu = 50$, $\sigma = 10$) avec `<MINTED>`. Tracez un histogramme et superposez la courbe théorique avec `<MINTED>`.

Exercice 9.4 (★★). Simulez une expérience : générez deux échantillons de taille 30, l'un de moyenne 100 et l'autre de moyenne 105 (même écart-type 15). Effectuez un test t. Répétez 1000 fois et comptez le pourcentage de fois où $p < 0.05$ (c'est la **puissance** du test).

Exercice 9.5 (★ ★ ★). Chargez le jeu de données intégré `<MINTED>` en R. Réalisez une analyse complète : statistiques descriptives par espèce, test ANOVA avec `<MINTED>` pour comparer les longueurs de sépales entre espèces, et régression linéaire entre longueur et largeur de pétales. Produisez 3 graphiques pertinents.

Fonctions clés		
Chapitre 9 — R : l'essentiel	Concept	Syntaxe R
	Affectation	<code><MINTED></code>
	Vecteur	<code><MINTED></code>
	Séquence	<code><MINTED></code> ou <code><MINTED></code>
	Data frame	<code><MINTED></code>
	Moyenne	<code><MINTED></code>
	Écart-type	<code><MINTED></code>
	Résumé	<code><MINTED></code>
	Test t	<code><MINTED></code>
	Corrélation	<code><MINTED></code>
	Régression	<code><MINTED></code>
	Graphique	<code><MINTED></code> , <code><MINTED></code> , <code><MINTED></code>

Chapitre 10

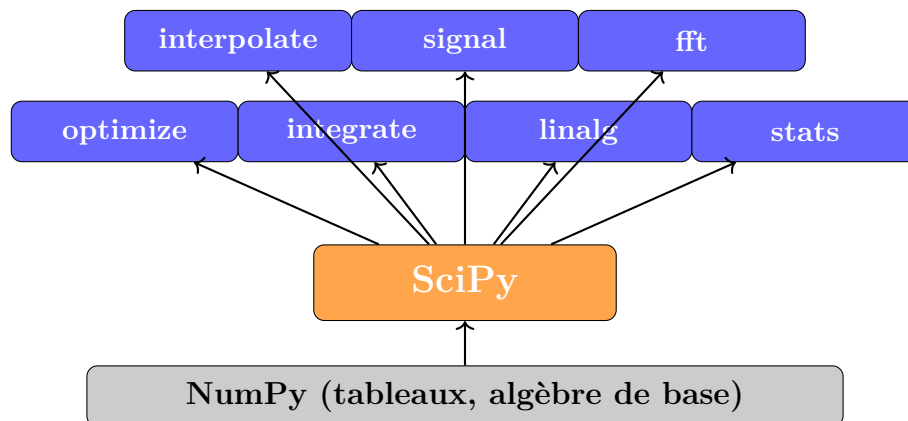
Calcul Scientifique avec SciPy

Intuition

NumPy fournit les briques de base (tableaux, algèbre linéaire élémentaire), mais le scientifique a souvent besoin d'outils plus spécialisés : ajuster une courbe à des données expérimentales, intégrer une fonction, résoudre un système d'équations ou effectuer des tests statistiques. C'est exactement le rôle de **SciPy**, une bibliothèque construite au-dessus de NumPy.

10.1 Vue d'ensemble de SciPy

Définition 10.1 (SciPy). **SciPy** (Scientific Python) est une bibliothèque open source qui fournit des algorithmes numériques efficaces organisés en sous-modules thématiques.



10.2 <MINTED> — Optimisation et ajustement

10.2.1 Ajuster une courbe expérimentale avec <MINTED>

En physique, on mesure souvent une décroissance exponentielle (radioactivité, décharge d'un condensateur). Ajustons un modèle $A(t) = A_0 e^{-\lambda t}$ à des données bruitées.

Python

<MINTED>

Sortie

```
A0 = 5.047, lambda = 0.308
```

Les valeurs retrouvées ($A_0 \approx 5.05$, $\lambda \approx 0.31$) sont proches des paramètres réels ($A_0 = 5$, $\lambda = 0.3$).

Python

```
<MINTED>
```

10.2.2 Minimisation avec <MINTED>

Python

```
<MINTED>
```

Sortie

```
Minimum en x = 3.3424
Valeur f(x) = -1.8770
```

10.3 <MINTED> — Intégration numérique

La fonction <MINTED> calcule une intégrale définie numériquement.

Définition 10.2 (Intégration numérique). L'intégration numérique (ou quadrature) approxime la valeur d'une intégrale définie $\int_a^b f(x) dx$ en évaluant f en un nombre fini de points.

Python

```
<MINTED>
```

Sortie

```
Integrale de sin(x) de 0 a pi = 2.000000
Valeur exacte = 2.000000
Integrale gaussienne = 1.772454
sqrt(pi) = 1.772454
```

10.4 <MINTED> — Algèbre linéaire

10.4.1 Résoudre un système linéaire

Considérons le système :

$$\begin{cases} 2x + y = 5 \\ x + 3y = 7 \end{cases} \iff \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 5 \\ 7 \end{pmatrix}$$

Python

<MINTED>

Sortie

Solution : $x = 1.60$, $y = 1.80$

10.4.2 Valeurs propres

Python

<MINTED>

Sortie

Valeurs propres : [1.38196601 3.61803399]
 Vecteur propre 1 : [-0.85065081 0.52573111]

10.5 <MINTED> — Statistiques

Python

<MINTED>

Sortie

Moyenne : 171.77
 Ecart-type : 10.14
 Test Shapiro-Wilk : $p = 0.8453$
 -> Distribution compatible avec la normalite

Python

<MINTED>

Sortie

t = -2.574, p-value = 0.0116

10.6 <MINTED> — Interpolation

L'interpolation permet d'estimer des valeurs entre des points de mesure.

Python

<MINTED>

Sortie

T a t=2.5 (lineaire) : 29.70 °C
 T a t=2.5 (cubique) : 30.22 °C

Python

<MINTED>

Ce qui peut mal tourner

Erreurs fréquentes avec SciPy

1. **Confondre <MINTED> et <MINTED>** — Les deux existent, mais <MINTED> est plus complet et plus optimisé. Préférez toujours <MINTED> pour les calculs sérieux.
2. **Oublier le point de départ dans <MINTED>** — La fonction <MINTED> nécessite un <MINTED>. Un mauvais choix initial peut conduire à un minimum *local* au lieu du minimum global.
3. **Mauvais modèle dans <MINTED>** — Si le modèle mathématique ne correspond pas aux données, l'ajustement sera mauvais. Vérifiez toujours visuellement le résultat.
4. **Ignorer la matrice de covariance** — <MINTED> retourne également <MINTED>, qui permet de calculer les incertitudes sur les paramètres ajustés : <MINTED>.

Mini-projet scientifique

Analyse et ajustement de données expérimentales Simulez une expérience de décharge d'un condensateur :

1. Générez des données bruitées suivant $V(t) = V_0 e^{-t/\tau}$ avec $V_0 = 12 \text{ V}$ et $\tau = 3 \text{ s}$ (ajoutez du bruit gaussien).
2. Utilisez <MINTED> pour retrouver V_0 et τ .
3. Calculez les incertitudes à partir de la matrice de covariance.
4. Intégrez numériquement l'énergie dissipée : $E = \int_0^{20} \frac{V(t)^2}{R} dt$ avec $R = 1 \text{ k}\Omega$.
5. Tracez les données, la courbe ajustée et les barres d'erreur.
6. Comparez le τ obtenu avec la valeur théorique.

10.7 Exercices

Exercice 10.1 (★). Calculez numériquement l'intégrale $\int_0^1 e^{-x^2} dx$ avec <MINTED>. Comparez avec la valeur approchée ≈ 0.7468 .

Exercice 10.2 (*). Résolvez le système linéaire : $3x + 2y - z = 1$, $2x - y + 4z = -2$, $x + y + z = 0$. Vérifiez en calculant $A \cdot x$ avec NumPy.

Exercice 10.3 (**). Générez 200 points suivant une loi normale ($\mu = 50$, $\sigma = 8$). Effectuez un test de Shapiro-Wilk et un test de Kolmogorov-Smirnov (<MINTED>) pour vérifier la normalité. Interprétez les p-valeurs.

Exercice 10.4 (**). Des données expérimentales suivent la loi $y = a \sin(bx + c)$. Générez des données avec $a = 3$, $b = 2$, $c = 0.5$ plus du bruit. Utilisez <MINTED> pour retrouver les paramètres. Attention : donnez des valeurs initiales raisonnables avec le paramètre <MINTED>.

Exercice 10.5 (***). Modélisez la cinétique d'une réaction chimique du second ordre : $\frac{d[A]}{dt} = -k[A]^2$. La solution analytique est $[A](t) = \frac{[A]_0}{1+k[A]_0 t}$. Générez des données bruitées, ajustez le modèle pour trouver k et $[A]_0$, puis intégrez numériquement $\int_0^{10} [A](t) dt$ (quantité totale restante). Comparez intégration numérique et solution analytique.

Fonctions clés		
Chapitre 10 — SciPy : l'essentiel	Sous-module	Fonctions clés
	<MINTED>	<MINTED>, <MINTED>
	<MINTED>	<MINTED> — intégrale définie
	<MINTED>	<MINTED>, <MINTED>
	<MINTED>	<MINTED>, <MINTED>, distributions
	<MINTED>	<MINTED>
	Astuces	
	Incertitudes fit	<MINTED>
	Bornes infinies	<MINTED>
	Valeurs initiales	<MINTED>

Chapitre 11

Bonnes Pratiques — Tests, Documentation, Git

Intuition

Écrire du code qui fonctionne est une chose ; écrire du code **fiable**, **lisible** et **re-productible** en est une autre. En science, un résultat doit être vérifiable et reproductible. Ce chapitre présente les outils et méthodes qui font la différence entre un script jetable et un projet scientifique de qualité : style de code, documentation, tests et contrôle de version.

11.1 Style de code : PEP 8

Définition 11.1 (PEP 8). **PEP 8** est le guide de style officiel de Python. Il définit des conventions de nommage, d'indentation et de mise en forme pour rendre le code lisible et cohérent.

Python

<MINTED>

Bonne pratique

Règles essentielles de PEP 8 :

- **Indentation** : 4 espaces (jamais de tabulations).
- **Longueur de ligne** : maximum 79 caractères.
- **Noms de variables** : <MINTED> (ex. : <MINTED>).
- **Noms de classes** : <MINTED> (ex. : <MINTED>).
- **Constantes** : <MINTED> (ex. : <MINTED>).
- **Espaces autour des opérateurs** : <MINTED>, pas <MINTED>.

Python

<MINTED>

11.2 Docstrings et documentation

Une **docstring** est une chaîne de documentation placée juste après la définition d'une fonction, d'une classe ou d'un module.

Python

<MINTED>

Python

<MINTED>

Sortie

```
Help on function energie_cinetique in module __main__:

energie_cinetique(masse, vitesse)
    Calcule l'energie cinetique d'un objet.

    Parameters
    -----
    masse : float
        Masse de l'objet en kilogrammes (kg).
    vitesse : float
        Vitesse de l'objet en metres par seconde (m/s).

    Returns
    -----
    float
        Energie cinetique en joules (J).
```

Remarque 11.2. Le style utilisé ci-dessus s'appelle le **style NumPy**. C'est le standard dans l'écosystème scientifique Python. D'autres styles existent (Google, Sphinx), mais le style NumPy est le plus répandu en sciences.

11.3 Tests avec <MINTED>

Tester son code, c'est comme vérifier une mesure expérimentale : on compare le résultat obtenu au résultat attendu.

Python

<MINTED>

Sortie

Tous les tests sont passes !

Python

<MINTED>

Sortie

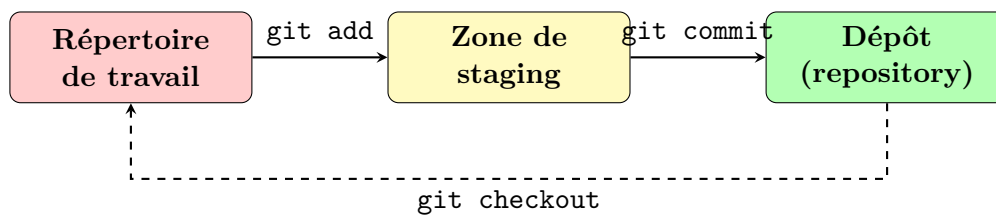
Tous les tests sont passes !

Attention

Pour les nombres à virgule flottante, ne testez *jamais* l'égalité exacte. Utilisez une tolérance : <MINTED> ou <MINTED>.

11.4 Contrôle de version avec Git

Définition 11.3 (Git). **Git** est un système de contrôle de version distribué. Il enregistre l'historique complet des modifications d'un projet, permettant de revenir en arrière, de collaborer et de travailler sur des branches parallèles.



Python

<MINTED>

Exemple 11.4 (Session Git typique).

Python

<MINTED>

11.5 Environnements virtuels

Définition 11.5 (Environnement virtuel). Un **environnement virtuel** est un répertoire isolé contenant une installation Python indépendante. Chaque projet peut ainsi avoir ses propres dépendances sans conflit avec d'autres projets.

Python

<MINTED>

Bonne pratique

Créez **toujours** un environnement virtuel pour chaque projet. Ajoutez le dossier de l'environnement (`mon_env/`) au fichier `.gitignore` pour ne pas le versionner.

11.6 Structure d'un projet scientifique

Python

<MINTED>

Python

<MINTED>

Ce qui peut mal tourner

Erreurs fréquentes en bonnes pratiques

1. **Ne pas tester son code** — « Ça a l'air de marcher » n'est pas un test. Même un simple <MINTED> vaut mieux que rien. Les erreurs silencieuses sont les plus dangereuses en science.
2. **Ne pas versionner** — Sans Git, une modification malheureuse peut détruire des heures de travail. Faites des commits **réguliers** avec des messages **descriptifs**.
3. **Noms de variables obscurs** — <MINTED>, <MINTED>, <MINTED> ne disent rien. Préférez <MINTED>, <MINTED>, <MINTED>.
4. **Code non documenté** — Dans six mois, vous ne vous souviendrez plus de ce que fait votre fonction. Ajoutez une docstring à chaque fonction.
5. **Tout dans un seul fichier** — Découpez votre projet en modules logiques. Un fichier de 2000 lignes est difficile à maintenir.

Mini-projet scientifique

Organiser un mini-projet scientifique Créez un projet complet en suivant les bonnes pratiques :

1. Créez la structure de répertoires décrite ci-dessus.
2. Initialisez un dépôt Git et créez un `.gitignore`.
3. Dans `src/analyse.py`, écrivez trois fonctions documentées (par exemple : chargement de données, calcul de statistiques, ajustement).
4. Dans `tests/test_analyse.py`, écrivez au moins 5 tests avec <MINTED>.
5. Créez un environnement virtuel et un `requirements.txt`.

6. Faites au moins 3 commits Git avec des messages clairs.
7. Vérifiez votre historique avec `git log --oneline`.

11.7 Exercices

Exercice 11.1 (*). Corrigez le code suivant pour respecter PEP 8. Renommez les variables, ajoutez des espaces et une docstring.

<MINTED>

Exercice 11.2 (*). Écrivez une fonction <MINTED> qui teste si un entier est premier. Ajoutez une docstring au style NumPy et au moins 5 tests avec <MINTED> (incluant les cas limites : 0, 1, 2, nombres négatifs).

Exercice 11.3 (**). Créez un module `physique.py` contenant trois fonctions documentées : <MINTED>, <MINTED> et <MINTED>. Écrivez un fichier de tests séparé vérifiant chaque fonction avec des valeurs connues.

Exercice 11.4 (**). Initialisez un dépôt Git dans un nouveau répertoire. Créez un fichier `calculs.py`, faites un premier commit. Modifiez le fichier, faites un deuxième commit. Utilisez `git diff HEAD~1` pour voir les différences. Utilisez `git log --oneline` pour afficher l'historique.

Exercice 11.5 (***). Reprenez l'un de vos projets des chapitres précédents (par exemple l'analyse de données Pandas ou l'ajustement SciPy). Réorganisez-le en projet structuré : séparez le code en modules, ajoutez des docstrings à toutes les fonctions, écrivez des tests, créez un environnement virtuel avec `requirements.txt`, et versionnez le tout avec Git (au moins 5 commits).

Fonctions clés

Chapitre	11	—	Bonnes	pratiques	:	l'essentiel
Pratique	Description					
PEP 8	Style officiel Python : <MINTED>, 4 espaces, 79 car.					
Docstrings	Documentation intégrée au code (style NumPy)					
<MINTED>	Vérification : <MINTED>					
<MINTED>	Comparaison flottante avec tolérance					
<code>git init</code>	Initialiser un dépôt					
<code>git add + commit</code>	Enregistrer des modifications					
<code>git log</code>	Consulter l'historique					
<code>python -m venv</code>	Créer un environnement virtuel					
<code>pip freeze</code>	Sauvegarder les dépendances					
<code>.gitignore</code>	Exclure des fichiers du suivi Git					

Référence rapide Python

.1 Types et opérations

Type	Description
<MINTEd>	Entier
<MINTEd>	Nombre à virgule flottante
<MINTEd>	Chaîne de caractères
<MINTEd>	Booléen (<MINTEd>/<MINTEd>)
<MINTEd>	Liste (modifiable)
<MINTEd>	Tuple (non modifiable)
<MINTEd>	Dictionnaire (clé-valeur)
<MINTEd>	Ensemble (sans doublons)

.2 Fonctions intégrées

Fonction	Description
<MINTEd>	Afficher
<MINTEd>	Longueur
<MINTEd>	Type d'un objet
<MINTEd>	Séquence d'entiers
<MINTEd>	Lire une entrée utilisateur
<MINTEd>, <MINTEd>, <MINTEd>	Conversion de type
<MINTEd>, <MINTEd>, <MINTEd>, <MINTEd>	Mathématiques de base
<MINTEd>, <MINTEd>	Tri et inversion
<MINTEd>, <MINTEd>	Itération avancée

.3 NumPy

Fonction	Description
<MINTEd>	Créer un tableau
<MINTEd>, <MINTEd>	Tableaux préremplis
<MINTEd>, <MINTEd>	Séquences
<MINTEd>, <MINTEd>, <MINTEd>	Fonctions mathématiques
<MINTEd>, <MINTEd>	Statistiques
<MINTEd>, <MINTEd>	Algèbre linéaire

.4 Référence rapide R

Fonction R	Description
<MINTED>	Créer un vecteur
<MINTED>, <MINTED>, <MINTED>	Statistiques
<MINTED>, <MINTED>	Tests statistiques
<MINTED>	Régression linéaire
<MINTED>, <MINTED>, <MINTED>	Graphiques
<MINTED>, <MINTED>	Données

Bibliographie

- [1] DOWNEY, A.B. (2024). *Think Python*. 3e éd., O'Reilly.
- [2] LANGTANGEN, H.P. (2016). *A Primer on Scientific Programming with Python*. 5e éd., Springer.
- [3] VANDERPLAS, J. (2016). *Python Data Science Handbook*. O'Reilly.
- [4] WICKHAM, H. & GROLEMUND, G. (2023). *R for Data Science*. 2e éd., O'Reilly.